

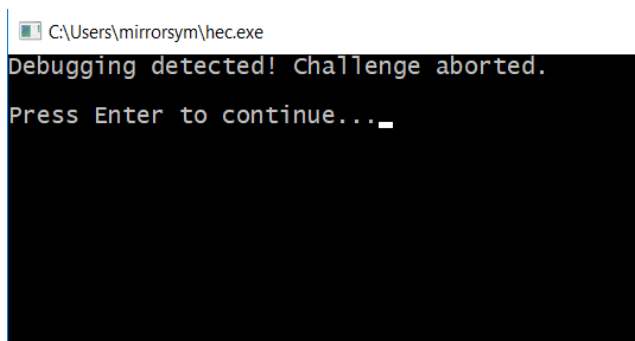
Hacker's Edge Book CrackMe v2

Author (write-up): msym

From crackmes.one (68bcfb41224c0ec5dcedbde2).

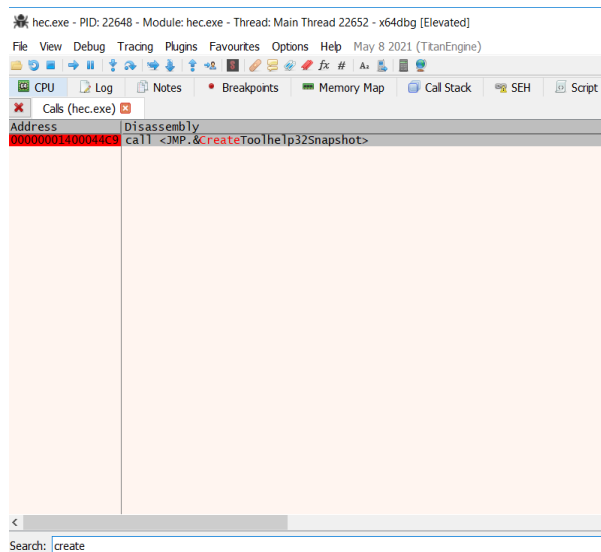
Part I: Bypassing anti-analysis checks

We start by launching the program (renamed `hec.exe`) and taking a look at how the user interface looks like. If x64dbg is running, we will be immediately greeted with the message in the screenshot below.

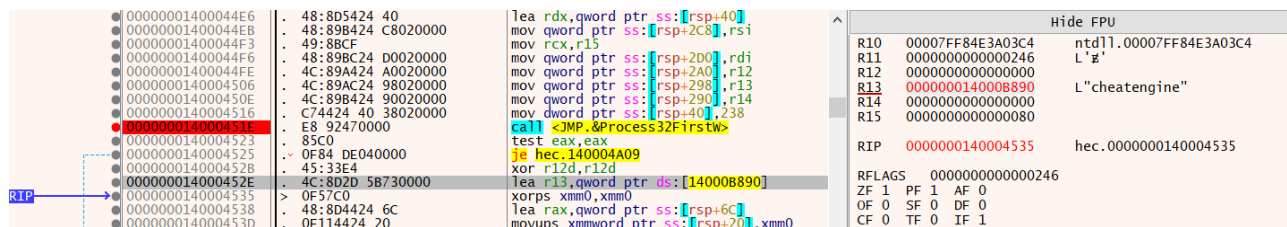


This immediately tells us that the program checks for processes that could reveal the presence of analysis tools. So instead of launching the program and attaching a debugger, we will need to launch the program within the debugger. We will be using x64dbg for this write-up. As we will also be using Ghidra later on, it is advisable to disable ASLR (for example, using CFF Explorer - `uncheck DLL can move under DllCharacteristics`).

Because we anticipate checks for running processes, the first thing we should do is to search for calls to APIs such as `CreateToolhelp32Snapshot` (used with `Process32First` and `Process32Next`). Indeed, inspecting intermodular calls with x64dbg, we notice these 3 APIs being called. To prevent the program from terminating, we will set a breakpoint at `CreateToolhelp32Snapshot` and try to circumvent detection. Hitting F9 once and we arrive at the call to `CreateToolhelp32Snapshot` .



If we continue to step over the next few instructions until 14000452E: `lea qword ptr ds: [14000B890]`, we will find a pointer to the string `cheatengine` placed in the R13 register.



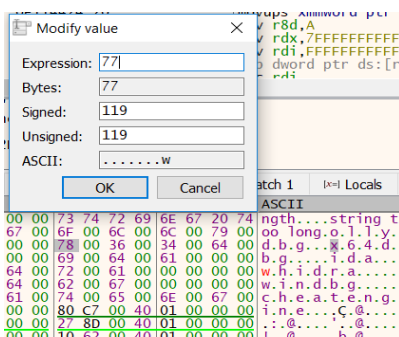
The program is very likely trying to determine if Cheat Engine is running. We can inspect 14000B890 in dump, upon which it will become clear which processes are being checked.

```

0000000014000B830 6E 67 74 68 00 00 00 00 73 74 72 69 6E 67 20 74 ngth....string t
0000000014000B840 6F 6F 20 6C 6F 6E 67 00 6F 00 6C 00 6C 00 79 00 oo long.o.l.l.y.
0000000014000B850 64 00 62 00 67 00 00 00 78 00 36 00 34 00 64 00 d.b.g...x.6.4.d.
0000000014000B860 62 00 67 00 00 00 00 00 69 00 64 00 61 00 00 00 b.g....i.d.a...
0000000014000B870 67 00 68 00 69 00 64 00 72 00 61 00 00 00 00 00 g.h.i.d.r.a....
0000000014000B880 77 00 69 00 6E 00 64 00 62 00 67 00 00 00 00 00 w.i.n.d.b.g....
0000000014000B890 63 00 68 00 65 00 61 00 74 00 65 00 6E 00 67 00 c.h.e.a.t.e.n.g.
0000000014000B8A0 69 00 6E 00 65 00 00 00 80 C7 00 40 01 00 00 00 i.n.e....Ç. @...

```

The screenshot above suggests that the program is checking for OllyDbg, x64dbg, IDA, Ghidra, WinDbg and Cheat Engine. One of the most expedient ways we can circumvent detection is to modify the strings in Dump. Because we have x64dbg and Ghidra running, we will replace the 'x' in x64dbg and the 'g' in ghidra with another ASCII character.



Once this is done, we can hit F9 once more, and we will be prompted to enter the solution.

0000000140007945	75 0D	jne hec.140007954
0000000140007947	E8 D4220000	call <JMP.&memcmp>
000000014000794C	85C0	test eax, eax
000000014000794E	75 04	jne hec.140007954
0000000140007950	B3 01	mov bl, 1
0000000140007952	EB 02	jmp hec.140007956
0000000140007954	32DB	xor bl, bl
0000000140007956	48:8D4C24 50	lea rcx, qword ptr ss:[rsp+50]
0000000140007958	E8 B08FFFFF	call <hec.sub_140003210>
0000000140007960	84DB	test bl, bl
0000000140007962	0F84 D5070000	je hec.14000813D
0000000140007968	0F57C0	xorps xmm0, xmm0
000000014000796B	0F114424 30	movups xmmword ptr ss:[rsp+30], xmm0
0000000140007970	48:C74424 40 0C000000	mov qword ptr ss:[rsp+40], C
0000000140007979	48:C74424 48 0F000000	mov qword ptr ss:[rsp+48], F
0000000140007982	F2:0F1005 E6490000	movsd xmm0, qword ptr ds:[14000C370]
000000014000798A	F2:0F114424 30	movsd qword ptr ss:[rsp+30], xmm0
0000000140007990	8B05 E2490000	mov eax, dword ptr ds:[14000C378]
0000000140007996	894424 38	mov dword ptr ss:[rsp+38], eax
000000014000799A	C64424 3C 00	mov byte ptr ss:[rsp+3C], 0
000000014000799F	4C:8D4C24 30	lea r9, qword ptr ss:[rsp+30]
00000001400079A4	41:B8 10000000	mov r8d, 10
00000001400079AA	48:8D15 273C0000	lea rdx, qword ptr ds:[14000B5D8]
00000001400079B1	48:8D8D B0020000	lea rcx, qword ptr ss:[rbp+2B0]
00000001400079B8	E8 43BFFFFF	call <hec.sub_140002D00>

Also notice there are other conditional jumps which will take us away from the function call at 1400079B8 . These are located at the addresses:

- 140007735 ;
- 1400077C1 ;
- 140007843 ;
- 1400078EA ; and
- 140007901 .

It will turn out that these other conditional jumps correspond merely to other checks (making sure input is not too long, checks for specific solutions which are incorrect), and the most important validation is the call to `memcmp` at 140007947 . The above observations give us an idea what and where to look out for. We can use Ghidra's decompiler to corroborate and/or speed up our analysis. We will give a very brief outline of what each of the checks corresponding to above conditional jumps likely do.

140007735 : Ensures that the length of the input string does not exceed 64 characters.

The length of the string is calculated within `sub_140001FF0` and held at `rbp+A0` (offset A0 from `rbp` after exiting `sub_140001FF0` , which Ghidra labelled as `local_368`). 8 is then subtracted and the result is compared with 0x38.

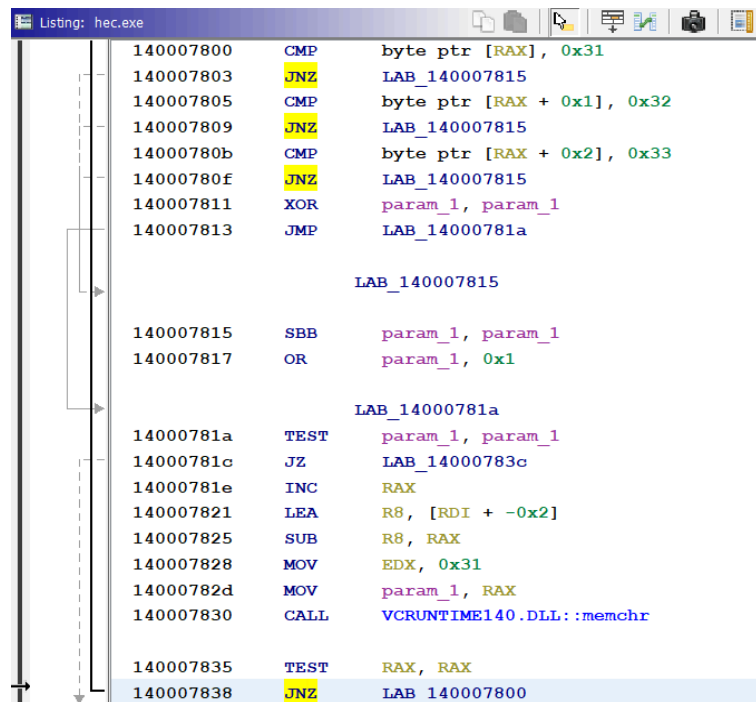
140007721	CALL	FUN_140001ff0
140007726	MOV	R15, qword ptr [RBP + local_368]
14000772d	LEA	RAX, [R15 + -0x8]
140007731	CMP	RAX, 0x38

1400077C1 : Rejects strings containing password as a substring.

```
if (7 < local_368) {
    for (pvVar12 = memchr(pbVar21,0x70,pbVar21 + uVar1 + (1 - pbVar21)); pvVar12 != 0x0;
        pvVar12 = memchr(pvVar12 + 1,0x70,pbVar21 + uVar1 + (1 - (pvVar12 + 1)))) {
        iVar6 = memcmp(pvVar12,"password",8);
        if (iVar6 == 0) {
            if (pvVar12 - pbVar21 != 0xffffffffffffffff) goto LAB_14000814b;
            break;
        }
    }
}
```

The program searches for the character p (charcode 0x70) with memchr , and where there's a hit, it uses memcmp to determine the presence of password . Note that the variable which Ghidra labelled as pbVar21 holds the address of the first character of the input string, so the condition pvVar12 - pbVar21 != -1 is trivially true (since pvVar12 >= pbVar21), so the jump is always taken whenever memcmp returns a match.

140007843 : Rejects solutions containing 123 as a substring. Ghidra's decompiler does not work very well and the assembly instructions offer more clarity.



```
Listing: hec.exe
140007800  CMP     byte ptr [RAX], 0x31
140007803  JNZ     LAB_140007815
140007805  CMP     byte ptr [RAX + 0x1], 0x32
140007809  JNZ     LAB_140007815
14000780b  CMP     byte ptr [RAX + 0x2], 0x33
14000780f  JNZ     LAB_140007815
140007811  XOR     param_1, param_1
140007813  JMP     LAB_14000781a

LAB_140007815
140007815  SBB     param_1, param_1
140007817  OR      param_1, 0x1

LAB_14000781a
14000781a  TEST    param_1, param_1
14000781c  JZ      LAB_14000783c
14000781e  INC     RAX
140007821  LEA     R8, [RDI + -0x2]
140007825  SUB     R8, RAX
140007828  MOV     EDX, 0x31
14000782d  MOV     param_1, RAX
140007830  CALL    VCRUNTIME140.DLL:memchr

140007835  TEST    RAX, RAX
140007838  JNZ     LAB_140007800
```

The program calls memchr to search for the character 1 (charcode 0x31), and for as long as there's a hit (i.e. zero flag is not set after 140007835: test rax, rax), the program compares the current byte and subsequent two bytes for 123 , and clears the RCX register (which Ghidra labelled as param_1 if 123 is found. If the RCX register is zero, a conditional jump takes the program to 14000783C and ultimately to 14000814B .

1400078EA : Rejects solutions containing admin as a substring.

```
if (4 < uVar4) {
    for (pvVar12 = memchr(pbVar21,0x61,pbVar21 + uVar4 + (-4 - pbVar21)); pvVar12 != 0x0;
        pvVar12 = memchr(pvVar12 + 1,0x61,pbVar21 + uVar4 + (-4 - (pvVar12 + 1)))) {
        lVar17 = 0;
        do {
            lVar18 = lVar17 + 1;
            if (*(pvVar12 + lVar17) != "admin"[lVar17]) {
                uVar7 = -(*(pvVar12 + lVar17) < "admin"[lVar17]) | 1;
                goto LAB_1400078be;
            }
            lVar17 = lVar18;
        } while (lVar18 != 5);
        uVar7 = 0;
LAB_100078be:
        if (uVar7 == 0) {
            if (pvVar12 - pbVar21 != 0xffffffffffffffff) goto LAB_14000814b;
            break;
        }
    }
}
```

The program searches for admin using memchr followed by memcmp , similar to the search for password .

140007901 : This checks the result of the call 1400078FA: call sub_1400050D0 . The function sub_1400050D0 appears to be checking for specific strings and rejecting them, including CRACK_ME_MASTER , DEBUG_MASTER_KEY and STATIC_ANALYSIS_WIN , among others. These strings are probably just distractors to make it difficult to find out which string is the actual solution.

RIP	0000000140005125	> 48:8BD3	mov rdx,rbx	rdx	Hide FPU
	0000000140005128	> 48:837B 18 10	cmp qword ptr ds:[rbx+18],10	rdx	
	000000014000512D	> 72 03	jb hec.140005132	rdx	RAX 0000445BE09AEC35
	000000014000512F	> 48:8B13	mov rdx,qword ptr ds:[rbx]	rdx	RBX 000000000254C6B0 "CRACK_ME_MASTER"
	0000000140005132	> 48:8BCF	mov rcx,rdi	rcx	RCX 000000000014E490 "mirrorsymmetry"
	0000000140005135	> 49:83FE 10	cmp r14,10	rdx	RDX 000000000254C6B0 "CRACK_ME_MASTER"
	0000000140005139	> 72 03	jb hec.14000513E	rcx	RBP 000000000254C790
	000000014000513B	> 48:8B0F	mov rcx,qword ptr ds:[rdi]	rcx	RSP 000000000014E270 "<?cinstd@3v?!
	000000014000513E	> 48:3B73 10	cmp rsi,qword ptr ds:[rbx+10]	rsi	RSP 000000000000000E "<?cinstd@3v?!
	0000000140005142	> 75 10	jne hec.140005154	rdi	RDI 000000000014E490 "mirrorsymmetry"
	0000000140005144	> 4C:88C6	mov r8,rsi		R8 0000000000000000
	0000000140005147	> E8 D44A0000	call <JMP.&memcmp>		R9 0000000000000000
	0000000140005148	> 85C0	test eax,ebx		R10 0000000000000010
	000000014000514E	> 0F84 8B000000	js hec.1400051D6		R11 000000000014E240
	0000000140005154	> 48:83C3 20	add rbx,20	rbx	R12 000000000014E840 "&"9a0b04c346db954e"
	0000000140005158	> 48:3BD0	cmp rbx,rbp		
	000000014000515B	> 75 C8	jne hec.140005125		

After clearing the above checks, we observe that the two strings which are being compared in the call to memcmp at 140007947 are 963393ee7749ee87 and 9a0b04c346db954e in the RCX and RDX registers respectively.

	000000014000790E	> 48:8D5424 50	lea rdx,qword ptr ss:[rsp+50]	rsi	Hide FPU
	0000000140007913	> 49:8BCC	mov rcx,r12	rcx	
	0000000140007916	> E8 D5C8FFFF	call <hec.sub_1400041F0>	rdx	RAX 000000000014E350 "&"963393ee7749ee87"
	0000000140007918	> 49:8BD4	mov rdx,r12	rdx	RBX 000000000014E490 "mirrorsymmetry"
	000000014000791E	> 49:837C24 18 10	cmp qword ptr ds:[r12+18],10	rdx	RCX 000000000254F090 "963393ee7749ee87"
	0000000140007924	> 72 04	jb hec.14000792A	rdx	RDX 000000000254F5D0 "9a0b04c346db954e"
	0000000140007926	> 49:8B1424	mov rdx,qword ptr ds:[r12]	rdx	RBP 000000000014E400
	000000014000792A	> 48:8D4C24 50	lea rcx,qword ptr ss:[rsp+50]	rsi	RSP 000000000014E300 "&"mirrorsymmetry"
	000000014000792F	> 48:837C24 68 10	cmp qword ptr ss:[rsp+68],10	rsi	RSP 7973726F7272696D
	0000000140007935	> 48:0F43C24 50	cmovae rcx,qword ptr ss:[rsp+50]	rsi	RDI 000000000014E49E
	000000014000793B	> 4C:8B4424 60	mov r8,qword ptr ss:[rsp+60]		R8 0000000000000010
	0000000140007940	> 4D:3B4424 10	cmp r8,qword ptr ds:[r12+10]		R9 0000000000000800
	0000000140007945	> 75 0D	jne hec.140007954		R10 000000000254DA20 "DET\x02"
	000000014000794C	> E8 D4220000	call <JMP.&memcmp>		R11 000000000014E080
	000000014000794E	> 85C0	test eax,ebx	eax	R12 000000000014E840 "&"9a0b04c346db954e"
	0000000140007950	> 75 04	jne hec.140007954		R13 000000000000000F
	0000000140007952	> EB 02	mov bl,1		R14 000000000014E496 "svmmetry"
	0000000140007954	> 32DB	xor bl,bl		

Also, the address (where 963393ee7749ee87 is held) in the RCX register is actually the second argument (passed via the RDX register) to the previous function call 1400051A7: call sub_1400041F0 . Furthermore, the third argument passed to the said function call is our input

string. It is very likely that `sub_1400041F0` is transforming our input string and the result is compared against `9a0b04c346db954e`. It is therefore probably worthwhile to take a look at `sub_1400041F0`.

```

0000000140004260 |> 4D:8BC1 | mov r8,r9
0000000140004263 |> 49:83FB 10 | cmp r11,10
0000000140004267 |> 72 03 | jb hec.14000426c
0000000140004269 |> 4D:8B01 | mov r8,qword ptr ds:[r9]
000000014000426C |> 48:C1CB 33 | ror rbx,33
0000000140004270 |> 0FB6CA | movzx ecx,d1
0000000140004273 |> 80E1 3F | and cl,3F
0000000140004276 |> 48:8BC7 | mov rax,rdi
0000000140004279 |> 48:D3E8 | shr rax,cl
000000014000427C |> 48:33D8 | xor rbx,rax
000000014000427F |> 49:0FBE0410 | movsx rax,byte ptr ds:[r8+rdx]
0000000140004284 |> 48:33D8 | xor rbx,rax
0000000140004287 |> 48:FFC2 | inc rdx
000000014000428A |> 49:3BD2 | cmp rdx,r10
000000014000428D |> 72 D1 | jb hec.140004260
000000014000428F |> 48:8D05 A2760000 | lea rax,qword ptr ds:[140008938]
0000000140004296 |> 48:894424 50 | mov qword ptr ss:[rsp+50],rax
0000000140004298 |> 48:8D05 9E760000 | lea rax,qword ptr ds:[140008940]
00000001400042A3 |> 48:8D4424 50 | mov qword ptr ss:[rsp+50],rax

```

r8: ^		r8:		Hide FPU	
RAX	0000000000000079				
RBX	963393EE7749EE87				'y'
RCX	000000000000000D				
RDX	000000000000000E				
RBP	0000000000014E1C0				
RSP	0000000000014E0C0				
RSI	0000000000014E290				
RDI	DEADBEEFCAFEBAE				
R8	0000000000014E490				"mirrorsymmetry"
R9	0000000000014E490				"mirrorsymmetry"
R10	0000000000000000E				
R11	0000000000000000F				
R12	0000000000014E840				
R13	0000000000000000F				&"9a0b04c346db954e"

From the screenshot above, it appears that the program contains a loop which transforms `DEADBEEFCAFEBAE` initially held in the `RBX` register into another 64-bit value via bitwise operations, which in particular involves a XOR operation with our input string, one character at a time. The value held in the `RBX` register right after the loop has terminated is `963393EE7749EE87`, which is precisely one of the strings `memcmp` is comparing as we've seen above.

We thus deduce that the the loop is likely implementing some sort of a hashing algorithm, and there does not appear to be a straightforward way of recovering the original string given its hash. For this crackme to be solvable, its creator probably left some clues behind elsewhere.

Part III: Finding the solution

Taking a look at the XREF for `FUN_1400041F0` in Ghidra, it seems that `FUN_1400041F0` is called at more than one address, and it is possible that program calculated the hash of the actual solution elsewhere.

The addresses other are:

- `140002BD7` ;
- `1400051A7` ;
- `140005EE8` ; and
- `140007916` .

It thus makes sense to set breakpoints in x64dbg at all of the above addresses to verify our hypothesis. Now we relaunch the crackme in x64dbg, and hit the breakpoint at (even before the call to `CreateToolhelp32Snapshot`). The second argument passed via the `RDX` register is an address, and the third argument passed via the `R8` register is (an address holding a pointer to) a string whose first three characters are `REV` .

0000000140002BC1	48:8D4D E0	lea rcx,qword ptr ss:[rbp-20]	[rt]		
0000000140002BC5	E8 06F6FFFF	call <hec.sub_1400021D0>			Hide FPU
0000000140002BCA	90	nop			
0000000140002BCB	4C:8D45 E0	lea r8,qword ptr ss:[rbp-20]	[rt]		RAX 000000000014E7B0 &"REV
0000000140002BCF	48:8D5424 40	lea rdx,qword ptr ss:[rsp+40]			RBX 0000000000000022 '..."
0000000140002BD4	48:8BCE	mov rcx,rsi			RCX 000000000014E840
0000000140002BD7	E8 14160000	call <hec.sub_1400041F0>			RDX 000000000014E710 "SOLU
0000000140002BDC	48:8BD8	mov rbx,rax	ra>		RBP 000000000014E7D0
0000000140002BDF	48:3BF0	cmp rsi,rax	ra>		RSP 000000000014E6D0
0000000140002BE2	74 62	je hec.140002C46			RSI 000000000014E840
0000000140002BE4	48:8B56 18	mov rdx,qword ptr ds:[rsi+18]	rd>		RDI 0000000000000023 '#'
0000000140002BE8	48:83FA 10	cmp rdx,10	rd>		
0000000140002BEC	72 2C	jnb hec.140002C1A			R8 000000000014E7B0 &"REV
0000000140002BEE	48:8B0E	mov rcx,qword ptr ds:[rsi]			R9 0000000000000003
0000000140002BF1	48:FFC2	inc rdx	rd>		R10 00007FF83EA70000 "MZÉ"
0000000140002BF4	48:81FA 00100000	cmp rdx,1000	rd>		R11 000000000014E610
0000000140002BF8	72 18	jnb hec.140002C15			R12 0000000000000000
0000000140002BFB	48:83C2 27	add rdx,27	rd>		R13 0000000000000000
0000000140002BFD	4C:8B41 F8	mov r8,qword ptr ds:[rcx-8]	r8>		R14 000000000014E7B0
0000000140002C01	49:7BC8	sub rcx,r8	r8>		

After the call to FUN_1400041F0 , the address passed as second argument is returned, which now contains (an address holding a pointer to) the hash 9a0b04c346db954e passed to memcmp at 140007947 .

0000000140002BC1	48:8D4D E0	lea rcx,qword ptr ss:[rbp-20]	[rt]		
0000000140002BC5	E8 06F6FFFF	call <hec.sub_1400021D0>			Hide FPU
0000000140002BCA	90	nop			
0000000140002BCB	4C:8D45 E0	lea r8,qword ptr ss:[rbp-20]	[rt]		RAX 000000000014E710 &"9a0b04c346db954e"
0000000140002BCF	48:8D5424 40	lea rdx,qword ptr ss:[rsp+40]	[rt]		RBX 0000000000000022 '..."
0000000140002BD4	48:8BCE	mov rcx,rsi	[rsi]		RCX 8B32B1173E510000 'd'
0000000140002BD7	E8 14160000	call <hec.sub_1400041F0>			RDX 0000000000000064
0000000140002BDC	48:8BD8	mov rbx,rax	ra>		RBP 000000000014E7D0
0000000140002BDF	48:3BF0	cmp rsi,rax	ra>		RSP 000000000014E6D0
0000000140002BE2	74 62	je hec.140002C46			RSI 000000000014E840
0000000140002BE4	48:8B56 18	mov rdx,qword ptr ds:[rsi+18]			RDI 0000000000000023 '#'
0000000140002BE8	48:83FA 10	cmp rdx,10			
0000000140002BEC	72 2C	jnb hec.140002C1A			R8 00000000002500DE0 "\fP\x02"
0000000140002BEE	48:8B0E	mov rcx,qword ptr ds:[rsi]			R9 00000000000008000
0000000140002BF1	48:FFC2	inc rdx			R10 0000000000251CAE0 &"a\rP\x02"
0000000140002BF4	48:81FA 00100000	cmp rdx,1000			R11 000000000014E450
0000000140002BF8	72 18	jnb hec.140002C15			R12 0000000000000000
0000000140002BFB	48:83C2 27	add rdx,27			R13 0000000000000000
0000000140002BFD	4C:8B41 F8	mov r8,qword ptr ds:[rcx-8]	r8>		R14 000000000014E7B0
0000000140002C01	49:7BC8	sub rcx,r8	r8>		

We therefore deduce that the aforementioned string with first three characters being REV is the solution to this crackme. Let's find out if our solution is correct.

```

=== Hacker's Edge - CrackMe Challenge ===
Author: Ryan Merket / 'webby'
Website: https://hackersedgebook.com

Find the hidden coupon code to unlock 100% off on Hacker's Edge!

Enter the solution: REV!

≡fÄë CONGRATULATIONS! ≡fÄë
You've successfully cracked the Hacker's Edge challenge!

Your 100% OFF coupon code:
Use this code on https://hackersedgebook.com for 100% off!

Bonus: You also found the 25% off coupon: HACKERS25OFF2025
Share this with friends who want to try the challenge!

Press Enter to continue...

```

NOTE: As the creator of the crackme seems to be offering a free hard copy of his memoir to those who manage to find the solution, the solution and the coupon code are redacted. The 25% coupon is available to everyone who attempted and is not redacted.

Last update: 14 January 2026